

Combo Partner API Documentation



Table of Contents

Table of Contents	1
Summary	3
Technical information	4
Concurrency	4
Idempotency	4
Limitations	4
Authorization	5
General HTTP Responses	6
Common use cases	7
Get and Update employee planning	7
Insert revenues	7
Get all employees and contracts information	7
Update personnel number for the payroll system for a given employee	8
Calculate Gross Salary for a Day	8
Import Postman Collection	9
Operations	
[CLIENT PARTNER]	11
List Client Partners	
[CONTRACT]	11
List Contracts	
[EMPLOYEE ID MAPPING]	13
List of Employee Id Mapping	16
Single Employee Id Mapping	18
Create Employee Id Mapping	20

Update Employee Id Mapping	
[LOCATION]	22
List Locations	24
[SHIFT]	
Planning	26
Single Shift	
[REST]	30
List Rests	
[REVENUE]	32
List Revenues	
[USER]	35
Create revenue	37
Single User	39
Definitions	41

Summary

API Version: 1

Last update: 2022-10-19

The endpoint to access the API is `https://partner.combohr.com`.

In this document we set up all technical documentation for our partner API, with generic json description and explicit examples will be found into the postman collection.

You may only get your own account information.

Endpoints are limited to 1.000 calls per minute. If the limit is exceeded, the IP is banned for 1 hour.

Technical information

Our application uses the standard ISO 8601 for datetime values. All dates, date times, times value must follow this convention or otherwise you may encounter unexpected behavior. Example: 2022-08-15.

Concurrency

The endpoints that are reading data are not blocking and are displaying the data available at the time the request was triggered. You should not see cases in which stale data is read, but keep in mind that there might be delay between the moment the update is made and when it's visible in a response. Usually this delay is 2 seconds.

Endpoints that are performing any kind of writes (creation/update) use optimistic locking.

Idempotency

All API endpoints are idempotent.

Limitations

The following limitations are in place at the time of writing this documentation. This is subject to change in the future and all changes will be reflected in this section.

- requests timeout after 90 seconds.

- it is mandatory to use encryption (HTTPS).

For further questions you can reach out at: support.partner@combohr.com

Authorization

In order to use our API you'll need an API token. This API token does not expire and cannot be regenerated.

If you want an API token refer to your current situation as :

I'm a client with CSM, then I just ask my CSM

I'm a client without CSM then I just ask the chat support

I'm not a client but I work for Combo client so I need to get by them (same as situation 1 or

2) I'm not a client then I can ask by the support email (support.partner@combohr.com)

Once you get the token you are ready to access the API.

In order to make API requests, you have to add an `Authorization` header. The header will look like this:

```
Authorization: Bearer [YOUR TOKEN HERE]
```

In case your token is invalid, the requests will have a response code of `401` .

General HTTP Responses

These responses can appear to any of the endpoints if not otherwise specified.

403

happening when you are not authorized to access the endpoint. This can happen because you don't have the necessary rights to access the resource/object or because you are trying to access information that is not available in your account.

the response body will contain error information. The error type will be `forbidden_action` and you will find specific information about the action and subject you try to access.

404

happening when the resource/object does not exist

the response body will contain error information. The error type will be `not_found` and a specific error message will be present.

500

happening in case something is wrong with the server

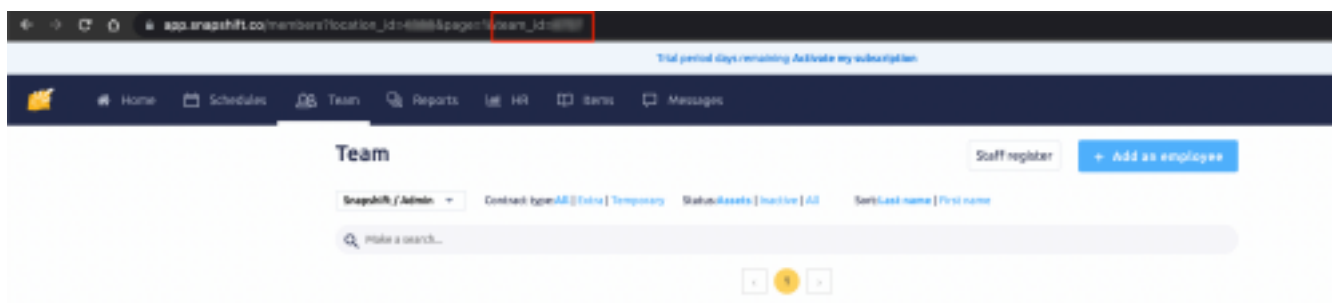
in case this response code is sent, please contact our support and describe the issue.

Common use cases

This section aims to provide some examples of use cases our API can answer. This list of examples are not exhaustive and intent to serve as guides through the discovery of our API.

Get and Update employee planning

1. get the authentication token for the account
2. get your location_id with operation "List Locations"
3. get your team_id by navigating to the "Team" tab and selecting a team.



4. list your plannings with operation "Planning" with the location_id, team_id and the time date you wish
5. from the responses extract the "id" information of the desired planning
6. get only the information of the desired planning by using operation "Single Planning" and the id you selected on previous step

At the moment we don't have the possibility to update an employee planning via API, this can be done only via the user interface.

Insert revenues

1. get the authentication token for the account
2. get your location_id with operation "List Locations"
3. insert revenue information using operation "Create revenue" with the location_id, the time date and values you wish

Get all employees and contracts information

1. get the authentication token for the account
2. get your location_id with operation "List Locations"
3. from the response select the "id" information of the desired location
4. list all contracts and employee information using the operation "List Contracts" with the location_id from the previous step

Update personnel number for the payroll system for a given employee

1. get the authentication token for the account
2. get your client_partner_id by using operation "List Client Partners" and filter with the name you wish
3. from the response select the "id" information of the desired client partner
4. get all the mappings available for your employees with operation "List of Employee Id Mappings" and the id from the previous step. (Tip: you can refine the search with the identification code you wish)
5. from the response extract the "id" information of the desired personnel number
6. use operation "Update Employee Id Mapping" with the id from the previous step and send the new value for identification_code

Calculate Gross Salary for a Day

1. get the authentication token for the account
2. get your location_id with operation "List Locations"
3. from the response select the "id" information of the desired location
4. get your planning information by using operation "Planning" with the location_id from the previous step and other filters of your choice
5. from the response select the "id" information of the desired shift
6. get information about your single shift by using operation "Single Shift" and the id from the previous step
7. from the response select the "real_starts_at", "real_ends_at", "real_break_duration", "firstname", "lastname" information
8. compute the hours the employee has worked with information from previous step ($\text{real_ends_at} - \text{real_starts_at} - \text{real_break_duration}$)
9. get the employee's contract information by using the operation "List Contracts" and filter by the same location_id and day as on step 3
10. from the response identify your employee's contract by its firstname and lastname and select "hourly_gross_rate" information
11. calculate the gross salary for the day with the information on step 8 and 10 ($\text{hours_worked} * \text{hourly_gross_rate}$)

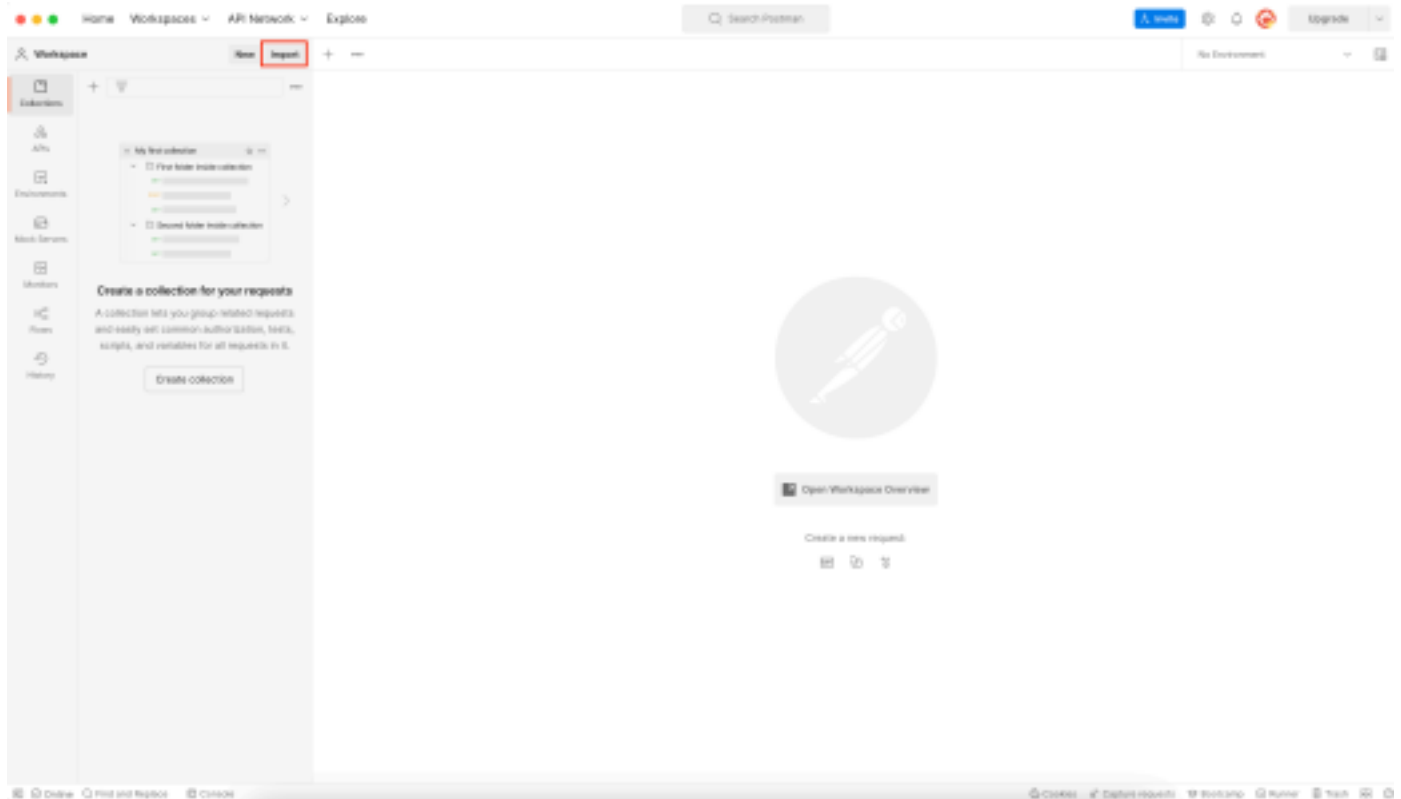
To find the calculations given in the analysis table, please refer to this article : [Le calcul de la masse salariale | Votre guide d'aide Combo](#)

Import Postman Collection

We use Postman to build and use our APIs. You can start getting familiar with our API by importing and trying out our postman collection.

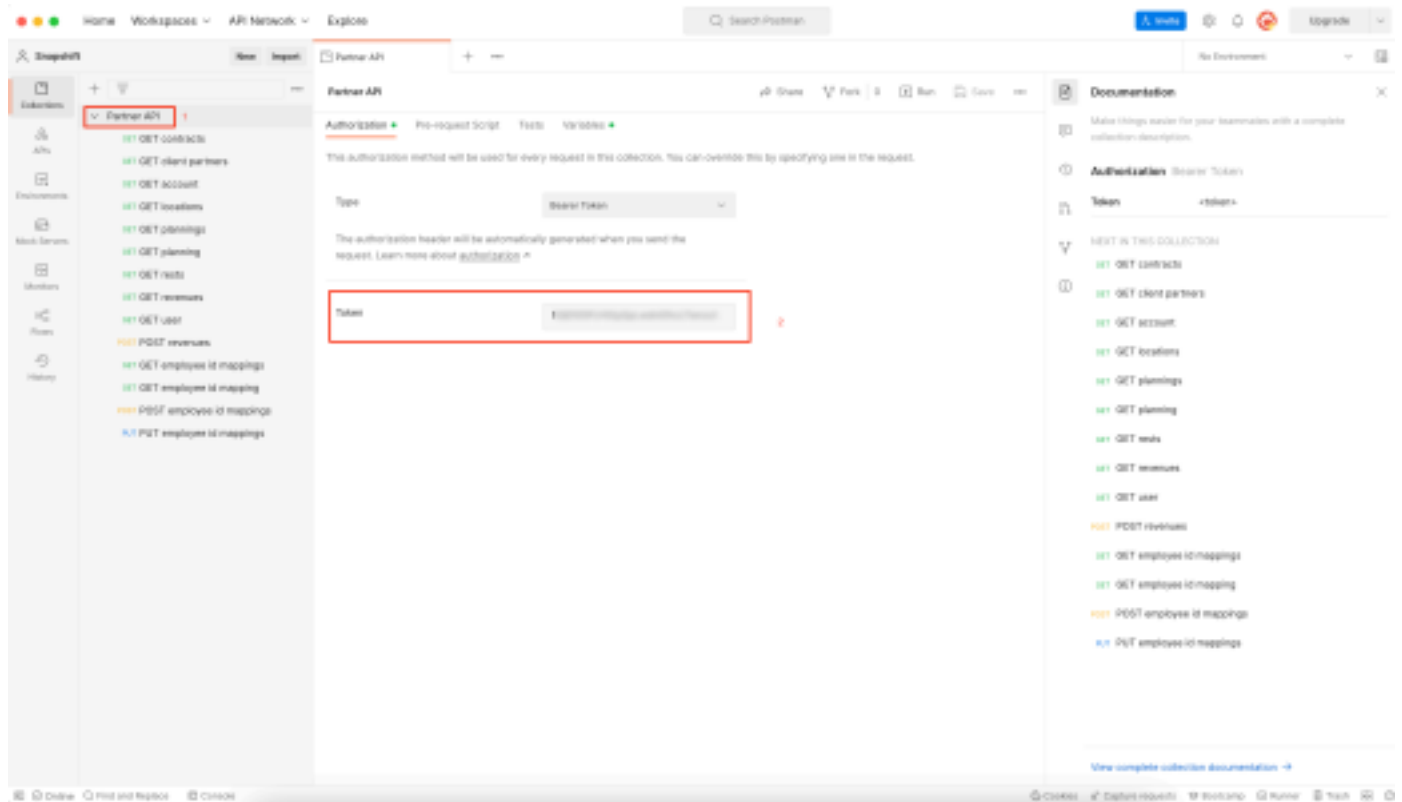
First of all, you have to install the Postman API platform on your system. You can easily download and install it from <https://www.postman.com/downloads/>.

Open the application and click the "Import" button, in the top left side of the screen.

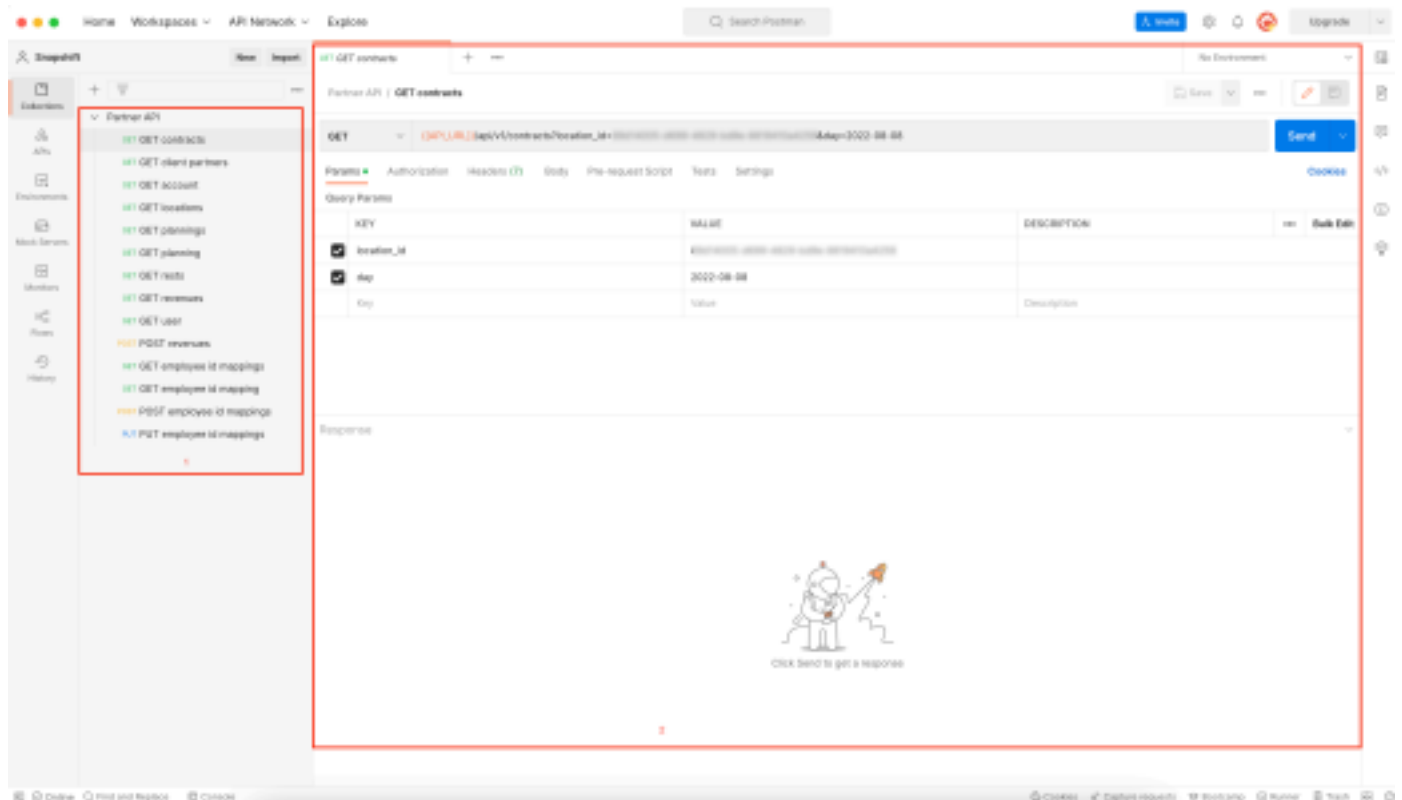


In the dialog that opens, upload the JSON postman collection file of our API and click the "Import" button.

Before using the APIs you have to configure the collection. Click on the collection name (1) and in the right side panel click "Authorization". Here you will have to fill in your API token (2).



On the left side of the screen you can see the list of available APIs (1). When you click on an API it will open on the right panel so you can see the Query params that can be customized or the request body. On the right panel you will also see the response of the API. (2)



You are ready now to explore our API.

Operations

List Client Partners

GET `/api/v1/client_partners?name={name}`

The request returns information about client partners. This endpoint is useful when you want to update or create a personnel number for all employees.

This endpoint can be filtered by name and return information for a single client partner. In case a client partner is not found with the given name, it will return an empty list.

List of partner personnel numbers ⓘ

Marceline Carrière	1676
SECRET GAYONE	3456
ADP	123
QW	Employee ID (ex: 123)
TEST	Employee ID (ex: 123)

Information

The maximum number of personnel numbers has been reached.

Parameters

parameter	type	required	default	description
name	string	no		The name of the client partner

HTTP Responses

Response Code: 200

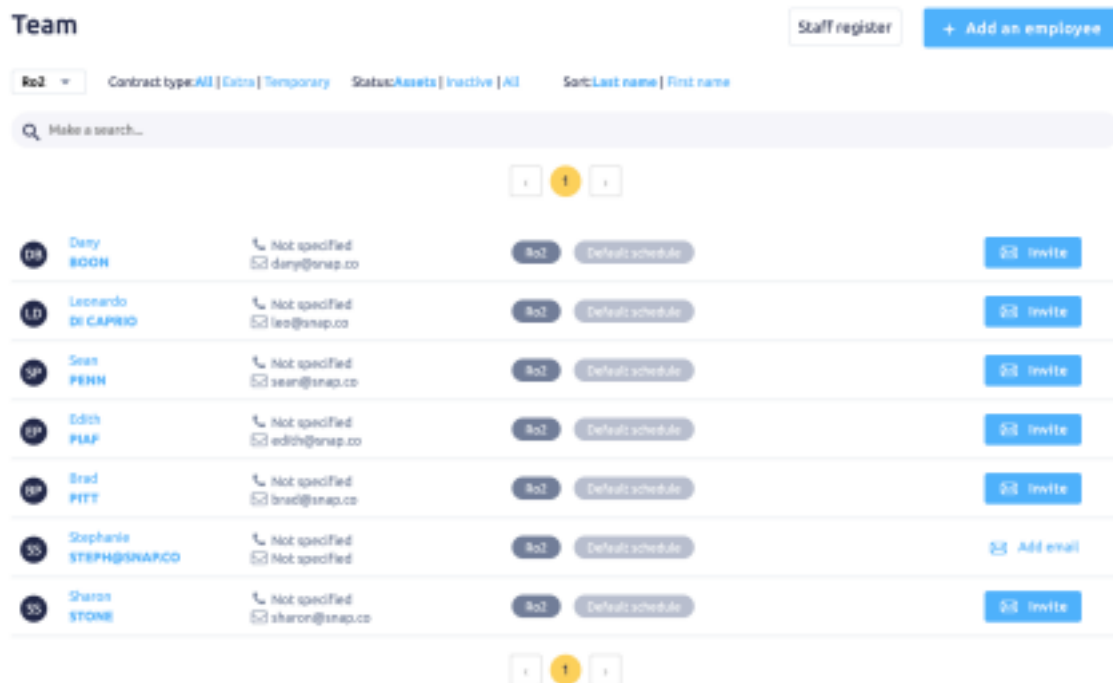
Response Body:
[

```
{
  "created_at": Datetime, the date and time when the client partner was
  added, "name": String, the name of the client partner,
  "updated_at": Datetime, the date and time when the client partner was
last updated,
  "id": String, the id of the client partner
},
...
]
```

List Contracts

GET `/api/v1/contracts?location_id={location_id}&day={day}`

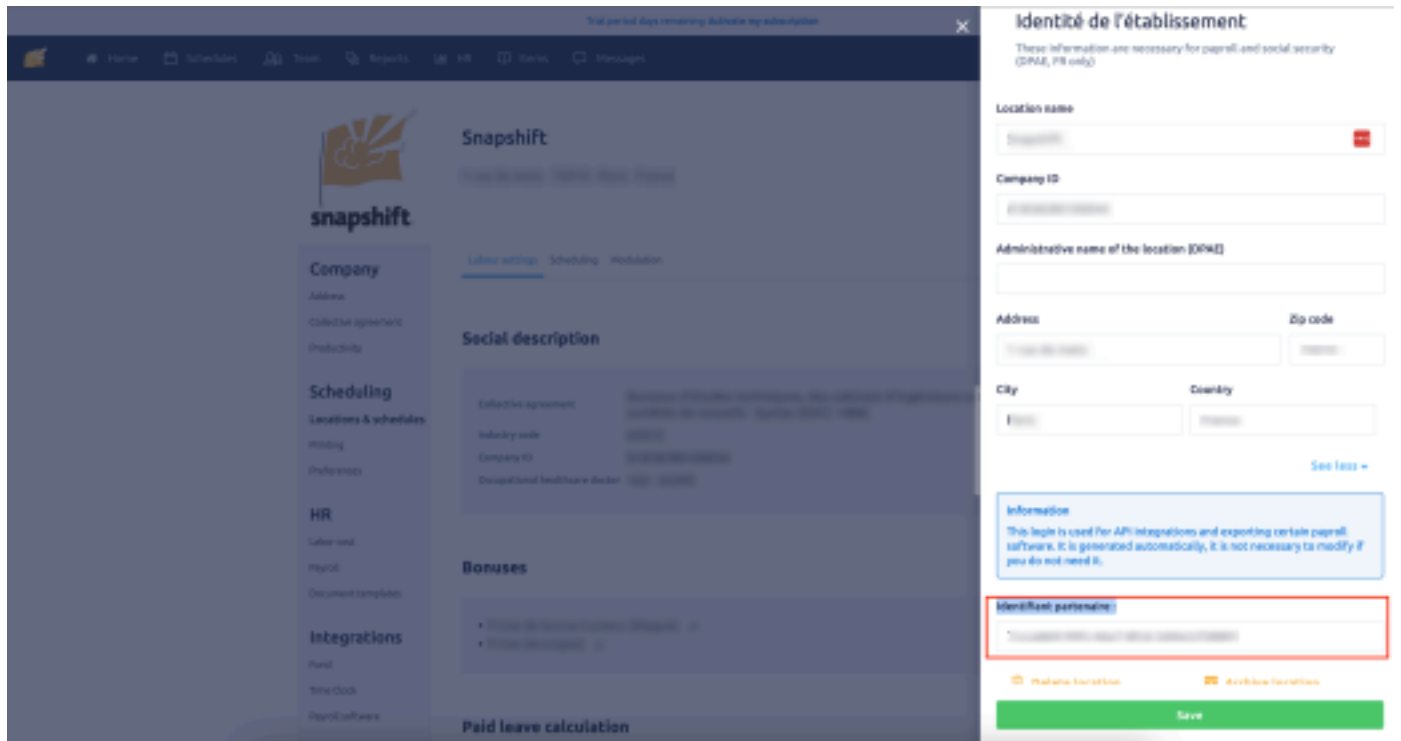
The request returns information about contracts for a given partner. This endpoint is useful to get basic information about all the contracts. The information can be filtered to see only contracts starting in the week of the date. Example: A contract starting on a wednesday (21 Sept) will be returned if the day filter is set to any value from that week (19-25 Sept). In case the location id is not found, then the response is an empty list.



Parameters

parameter	type	required	default	description
location_id	string	yes		The partner id of the location of the contract.
day	string	no	current date	Date in the ISO 8601 format. Will return contracts who are in progress on that day.

You can find `location_id` by clicking on the top right on your user name, then going to "Settings", then from the left menu select "Locations & schedules". From here click on one of your locations. Then click "Change" on the top right. In the form that opens click "See more". Your `location_id` is the string under "Identifiant partenaire :"



HTTP Responses

Response Code: 200

Response Body:

```
[
{
  "id": String, the partner id of the contract,
  "firstname": String, the user's first name,
  "lastname": String, the user's last name,
  "social_security_number": String, the user's social security number,
  "contract_time": Integer, the period of time the user will work per
  week, "contract_type": String, the contract type,
  "location_id": String, the location's partner_id,
  "role": String, the users's membership's role,
  "hourly_gross_rate": Float, the user's hourly gross rate,
  "start_date": Date, the user's start date,
  "end_date": Date, the user's end date,
  "working_days_in_week": Integer, the number of days in a week during which the
  user should have planned activities,
  "email": String, the user's email address,
  "street_address": String, the user's address,
  "zip": String, the user's zip code,
  "city": String, the user's city,
  "citizenship": String, the user's citizenship,
  "function": String, the user's function,
  "mobile_phone": String, the user's mobile phone number,
```

```
    "fixed_line_phone": String, the user's fixed phone number
  },
  ...
]
```

Response Code: 422

This can happen when you don't send all the mandatory parameters. In case you don't send location_id you will see this response code.

Response Body:

```
{
  "raw_error_messages": [
    "param is missing or the value is empty: location_id"
  ],
  "error": {
    "type": "missing_request_parameter",
    "parameter": "location_id",
    "codes": []
  },
  "message": "Missing parameter \"location_id\"."
}
```

Know bugs:

Response Code: 500

This can happen in case the day parameter does not respect the ISO 8601 format.

List of Employee Id Mapping

GET /api/v1/client_partners/{client_partner_id}/employee_id_mappings?identification_code={identification_code}

The request returns information about the partner personnel numbers of the client partner. This is used to get the needed identification for all the employees. The endpoint can be filtered by identification_code.

List of partner personnel numbers ⓘ

	Employee ID (ex: 123)
	123
	Employee ID (ex: 123)
	Employee ID (ex: 123)
	Employee ID (ex: 123)
	Employee ID (ex: 123)

Information
The maximum number of personnel numbers has been reached.

Parameters

parameter	type	required	default	description
client_partner_id	string	yes		The partner_id of the client partner.
identification_code	string	no		The identification code of a client partner

The client_partner_id can be found by using operation "List Client Partners" and selecting the attribute id.

HTTP Responses

Response Code: 200

Response Body:

```
[
  {
    "created_at": Datetime, the time and day when the resource was
```

```
created, "identification_code": String, the identification code,
"membership_id": Integer, the membership id,
"updated_at": Datetime, the time and day when the resource was last
updated, "id": String, the partner id,
"client_partner_id": String, the client partner id,
"client_partner_name": String, the client partner name
},
...
]
```

Single Employee Id Mapping

GET `/api/v1/client_partners/{client_partner_id}/employee_id_mappings/{id}`

The request returns information about the partner personnel numbers of the client partner's employee. This is used to get the needed identification for a single employee.

List of partner personnel numbers ⓘ

Employee ID	Employee ID (ex: 123)
ID	123
Employee ID	Employee ID (ex: 123)
ID	Employee ID (ex: 123)
Name (ex: Payfit)	Employee ID (ex: 123)

Information

The maximum number of personnel numbers has been reached.

Parameters

parameter	type	required	default	description
client_partner_id	string	yes		The partner_id of the client partner.
id	string	yes		The partner_id of the employee id mapping.

You can get your client_partner_id by using operation "List Client Partners" and filter with the name you wish

HTTP Responses

Response Code: 200

Response Body:

```
{
  "created_at": Datetime, the time and day when the resource was
  created, "identification_code": String, the identification code,
```

```
"membership_id": Integer, the membership id,  
"updated_at": Datetime, the time and day when the resource was last  
updated, "id": String, the partner id,  
"client_partner_id": String, the client partner id,  
"client_partner_name": String, the client partner name  
}
```

Create Employee Id Mapping

POST `/api/v1/client_partners/{client_partner_id}/employee_id_mappings`

The request creates information about the partner personnel numbers of the client partner's employee. The information will be created for the specified partner.

List of partner personnel numbers ⓘ

	Employee ID (ex: 123)
	123
	Employee ID (ex: 123)
	Employee ID (ex: 123)
Name (ex: Payfit)	Employee ID (ex: 123)

Information
The maximum number of personnel numbers has been reached.

Parameters

parameter	type	required	default	description
client_partner_id	string	yes		The partner_id of the client partner.

You can get your client_partner_id by using operation "List Client Partners" and filter with the name you wish

Payload

```
{
  "membership_id": Integer (required), the id of the membership,
  "identification_code": String (required), the identification code, free
text }
```

HTTP Responses

Response Code: 200

Response Body:

```
{  
  "created_at": Datetime, the time and day when the resource was  
  created, "identification_code": String, the identification code,  
  "membership_id": Integer, the membership id,  
  "updated_at": Datetime, the time and day when the resource was last  
  updated, "id": String, the partner id,  
  "client_partner_id": String, the client partner id,  
  "client_partner_name": String, the client partner name  
}
```

Update Employee Id Mapping

PUT /api/v1/client_partners/{client_partner_id}/employee_id_mappings/{id}

The request updates information about the partner personnel numbers of the client partner's employee. The information will be created for the specified partner.

List of partner personnel numbers ⓘ

XXXXXXXXXX	Employee ID (ex: 123)
123	123
XXXXXXXXXX	Employee ID (ex: 123)
123	Employee ID (ex: 123)
Name (ex: Payfit)	Employee ID (ex: 123)

Information

The maximum number of personnel numbers has been reached.

Parameters

parameter	type	required	default	description
client_partner_id	string	yes		The partner_id of the client partner.
id	string	yes		The id of the employee id mapping.

You can get your client_partner_id by using operation "List Client Partners" and filter with the name you wish

Payload

```
{
  "identification_code": String (required), the identification code, free
text }
```

HTTP Responses

Response Code: 200

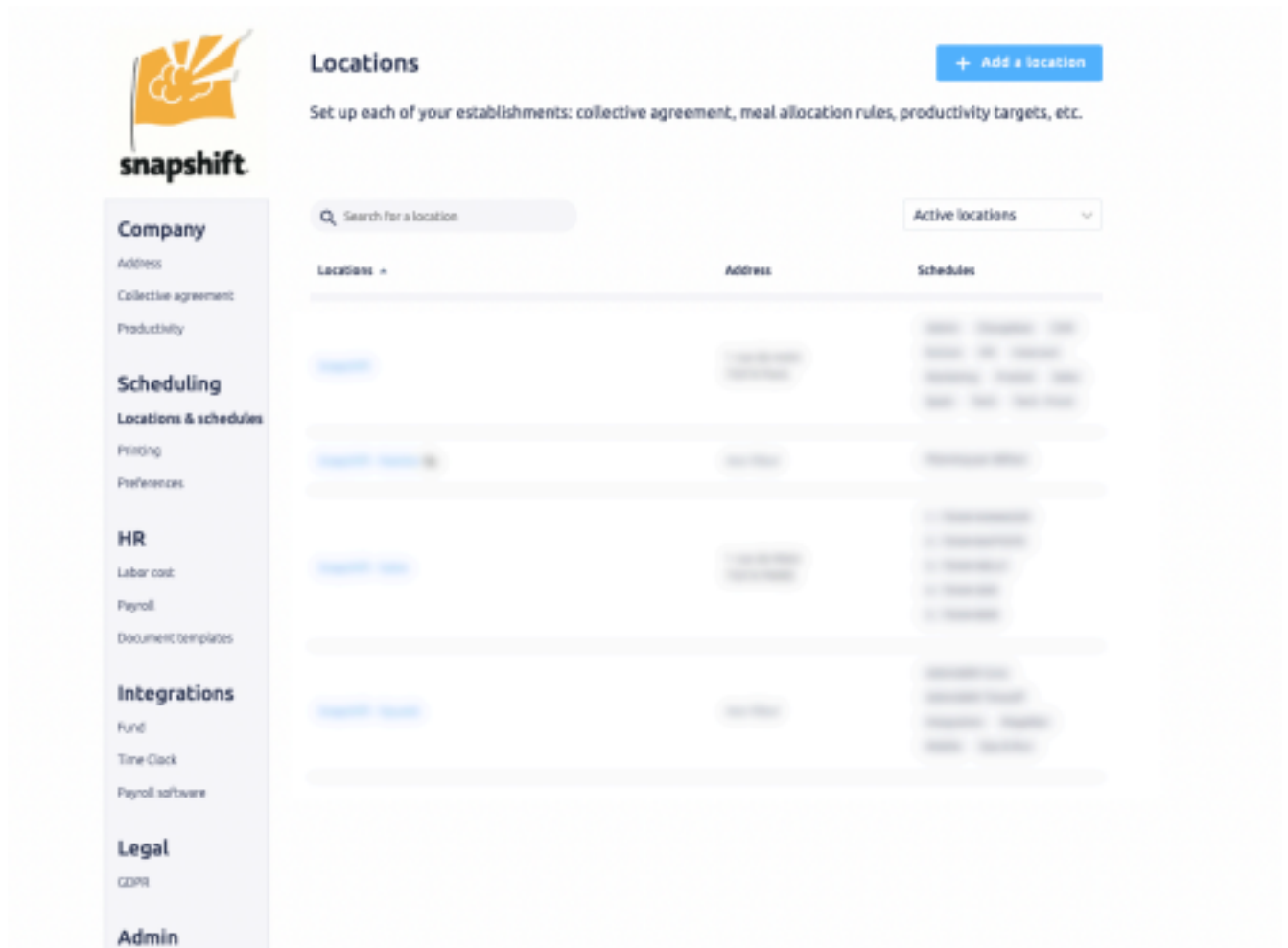
Response Body:

```
{
  "created_at": Datetime, the time and day when the resource was
  created, "identification_code": String, the identification code,
  "membership_id": Integer, the membership id,
  "updated_at": Datetime, the time and day when the resource was last
  updated, "id": String, the partner id,
  "client_partner_id": String, the client partner id,
  "client_partner_name": String, the client partner name
}
```

List Locations

GET `/api/v1/locations`

The request returns the locations of the account with their nested teams when available. This is used when we need information about all locations of the account.



HTTP Responses

Response Code: `200`

Response Body:

```
[
  {
    "id": String, the partner id,
    "name": String, the location's name,
    "account_id": String, the account's partner id,
    "partner_id": String, the partner id,
    "snapshot_account_id": Integer, the account's id,
    "snapshot_location_id": Integer, the location's id,
    "teams": Array[team], list of teams
  },
]
```

```
...  
]
```

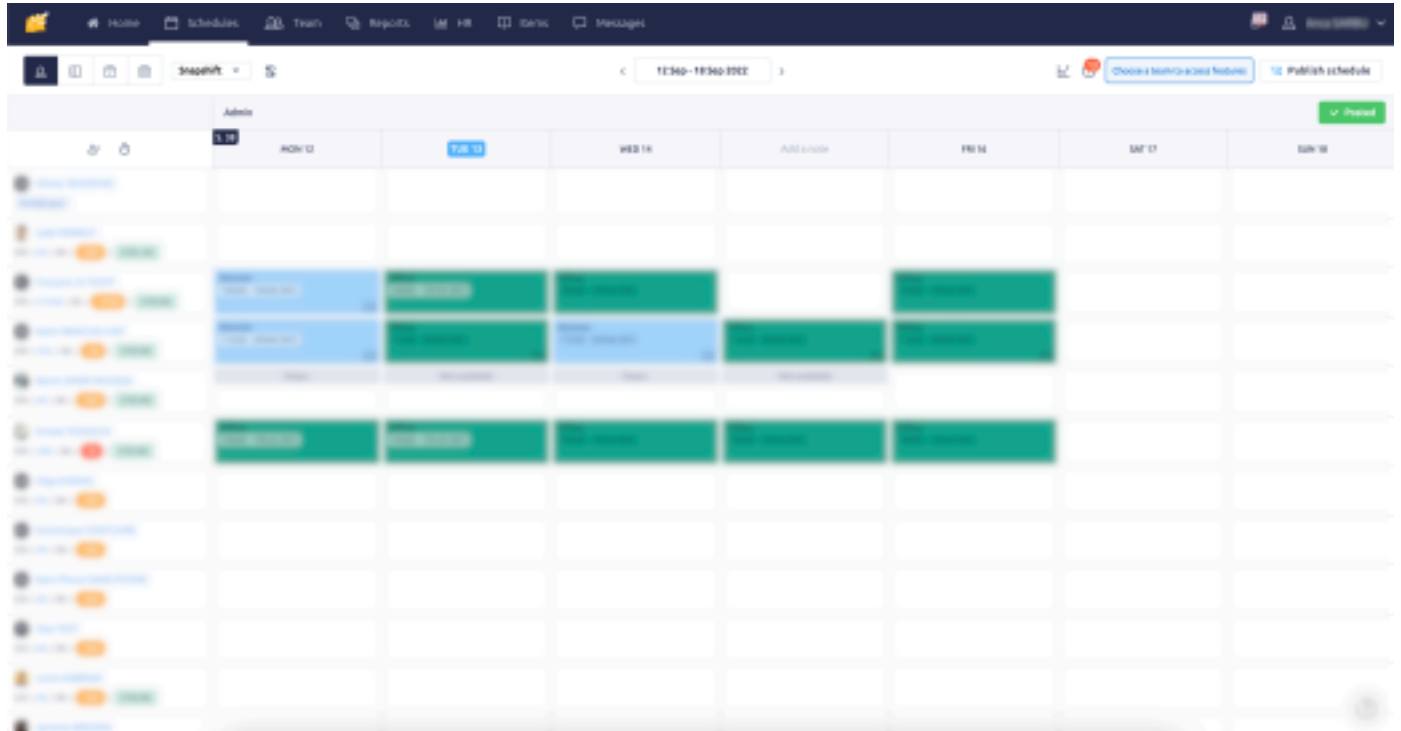
The teams attribute will be an empty object in case the location has just a single team. For more than 2 teams for a location, the information will be rendered by the response.

Planning

GET

```
/api/v1/plannings?start_date={start_date}&location_id={location_id}&team_id={team_id}&label_name={label_name}&end_date={end_date}
```

The request returns only the shifts of published schedules. This is used to get the schedule for a given period. The information can be filtered by location, team, label. If the planning is not published, an empty array will be returned.



Parameters

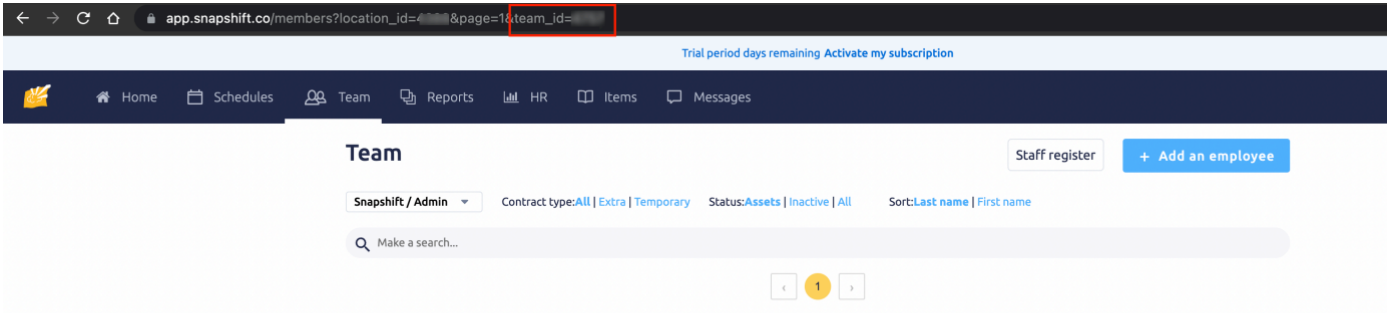
parameter	type	required	default	description
start_date	string	yes		Date in the ISO 8601 format, representing the start of the interval.
location_id	string	no		The partner id of the location of the contract.
team_id	string	no		The partner id of the team.
label_name	string	no		The name of the label.

end_date	string	no	start_date + 7 day	Date in the ISO 8601 format, representing the end of the interval. The maximum size of end_date - start_date period is 40 days. If end_date is more than 40 days later than start_date , no error message will be sent and API will return the data for 40 days after start_date .
----------	--------	----	-----------------------	--

You can find location_id by clicking on the top right on your user name, then going to "Settings", then from the left menu select "Locations & schedules". From here click on one of your locations. Then click "Change" on the top right. In the form that opens click "See more". Your location_id is the string under "Identifiant partenaire :"



You can find team_id by navigating to the "Team" tab and selecting a team.



HTTP Responses

Response Code: 200

Response Body:

```
[
  {
    "id": String, the partner id,
    "date": Date, the shift's start date,
    "starts_at": Datetime, the shift's planned start time, # TBD
    "ends_at": Datetime, the shift's planned end time, # TBD
    "break_duration": Integer, the shift's planned break time, # TBD
    "real_starts_at": Datetime, the shift's real start time, # TBD
    "real_ends_at": Datetime, the shift's real end time, # TBD
    "real_break_duration": Integer, the shift's real break time, # TBD
    "firstname": String, the employee's first name,
    "lastname": String, the employee's first name,
    "contract_id": String, the employee's contract partner id,
    "employee_number": String, the employee's number,
    "location_name": String, the location's name,
    "location_id": String, the location's partner id,
    "label_name": String, the label's name,
    "team_name": String, the team's name,
    "team_id": String, the team's partner id,
    "identification_codes": Array[identification_code], list of employee id
mapping objects
  },
  ...
]
```

Response Code: 422

This can happen when you don't send all the mandatory parameters. In case you don't send start_date you will see this response code.

Response Body:

```
{
  "raw_error_messages": [
    "param is missing or the value is empty: start_date"
  ],
  "error": {
    "type": "missing_request_parameter",
    "parameter": "start_date",
    "codes": []
  },
  "message": "Missing parameter \"start_date\"."
}
```

Known bugs:

Response Code: 500

This can happen in case the start_date or end_date parameter does not respect the ISO 8601 format.

Single Shift

GET /api/v1/plannings/{id}

Request returns the information for a given shift, including the events. This can be used to see details of the shift like who did the shift, when it started and ended, where the shift was scheduled, etc.

Validate shift

FL

Example of Shift

Tuesday 13 September

Scheduled

10:00 - 19:30 (90)

Pause : 90 min

Actual hours

Did not show

Label

Office

[Customize labels](#)

Hours

from 10:00

to 19:30

Break(s)

Unpaid

90

min

+ Indicate hours

[+ Add a break](#)

Meal

[+ Add meal](#)

Comments

Delete

Validate

Parameters

parameter	type	required	default	description
id	integer	yes		The id of the shift.

HTTP Responses

Response Code: 200

Response Body:

```
{
  "id": String, the partner id,
  "date": Date, the shift's start date,
  "starts_at": Datetime, the shift's planned start time, # TBD
  "ends_at": Datetime, the shift's planned end time, # TBD
  "break_duration": Integer, the shift's planned break time, #
  TBD "real_starts_at": Datetime, the shift's real start time, #
  TBD
  "real_ends_at": Datetime, the shift's real end time, # TBD
  "real_break_duration": Integer, the shift's real break time, #
  TBD "firstname": String, the employee's first name,
  "lastname": String, the employee's first name,
  "contract_id": String, the employee's contract partner id,
  "employee_number": String, the employee's number,
  "location_name": String, the location's name,
  "location_id": String, the location's partner id,
  "label_name": String, the label's name,
  "team_name": String, the team's name,
  "team_id": String, the team's partner id,
  "identification_codes": Array[identification_code], list of employee id mapping
  objects,
  "events": Array[event], list of event objects
}
```

Response Code: 200

When a shift with the given id doesn't exist.

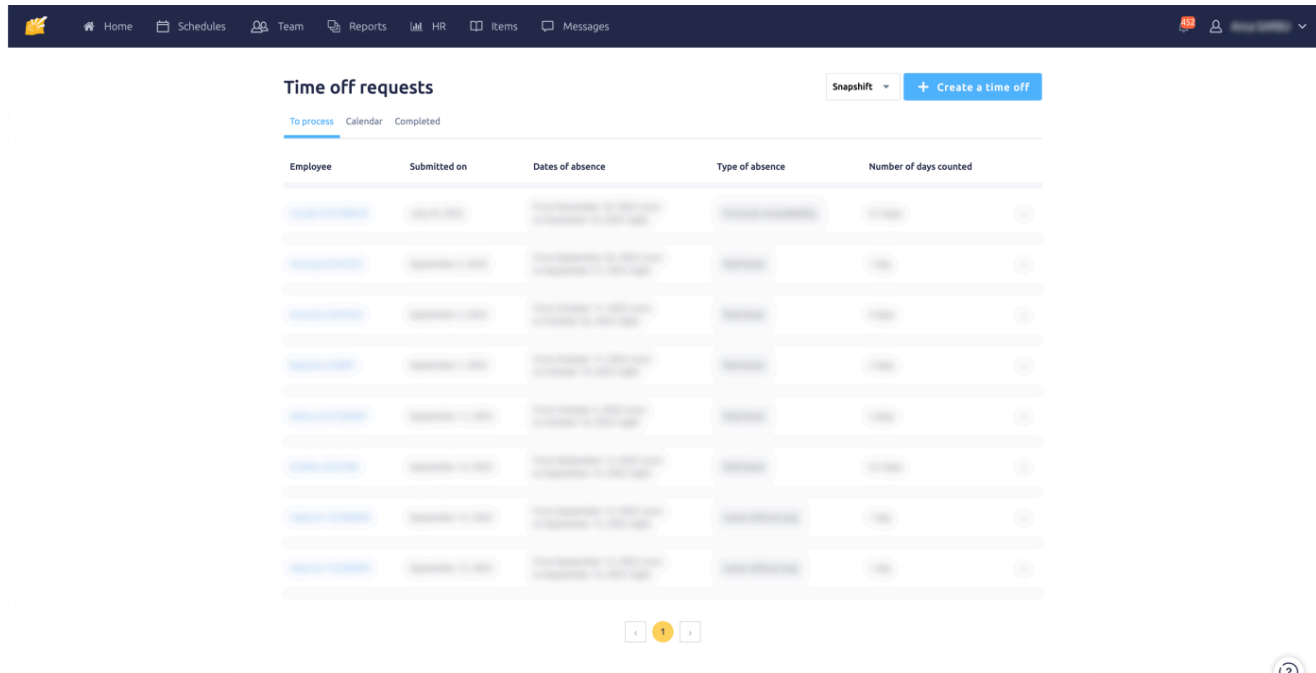
Response Body:

```
null
```

List Rests

GET `/api/v1/rests?start_date={start_date}&location_id={location_id}&end_date={end_date}`

Request returns the planning information of leaves, this includes any type of leave. The information can be filtered by location and period. It can be used to get all the absences in a given period of time.



Parameters

parameter	type	required	default	description
start_date	string	yes		Date in the ISO 8601 format, representing the start of the interval.
location_id	string	yes		The partner id of the location of the contract.
end_date	string	no	start_date + 7 days	Date in the ISO 8601 format, representing the end of the interval.

You can find `location_id` by clicking on the top right on your user name, then going to "Settings", then from the left menu select "Locations & schedules". From here click on one of your locations. Then click "Change" on the top right. In the form that opens click "See more". Your `location_id` is the string under "Identifiant partenaire :"

The screenshot displays the Snapshift web application interface. On the left is a sidebar with navigation links: Company, Scheduling, HR, and Integrations. The main content area is titled 'Identité de l'établissement' (Establishment Identity) and includes a sub-header stating 'These information are necessary for payroll and social security (DPAE, FR only)'. The form contains several input fields: 'Location name', 'Company ID', 'Administrative name of the location (DPAE)', 'Address', 'Zip code', 'City', and 'Country'. A red rectangular box highlights the 'Identifiant partenaire' field. At the bottom of the form, there are buttons for 'Update location' and 'Archive location', and a green 'Save' button.

HTTP Responses

Response Code: 200

Response Body:

```
[
  {
    "id": String, the partner id,
    "contract_id": String, the user's contract's partner id,
    "location_id": String, the location's partner id,
    "rest_type": String, the type of absence,
    "starts_at": Datetime, absence starting time,
    "ends_at": Datetime, absence end time,
    "custom_value": Array[custom_week_value], a list of objects containing custom
    week information # TBD
  },
  ...
]
```

Response Code: 422

This can happen when you don't send all the mandatory parameters. In case you don't send location_id or start_date you will see this response code.

Response Body:

```
{
  "raw_error_messages": [
    "param is missing or the value is empty: start_date"
  ]
}
```

```
],
  "error": {
    "type": "missing_request_parameter",
    "parameter": "start_date",
    "codes": []
  },
  "message": "Missing parameter \"start_date\"."
}
```

Known bugs:

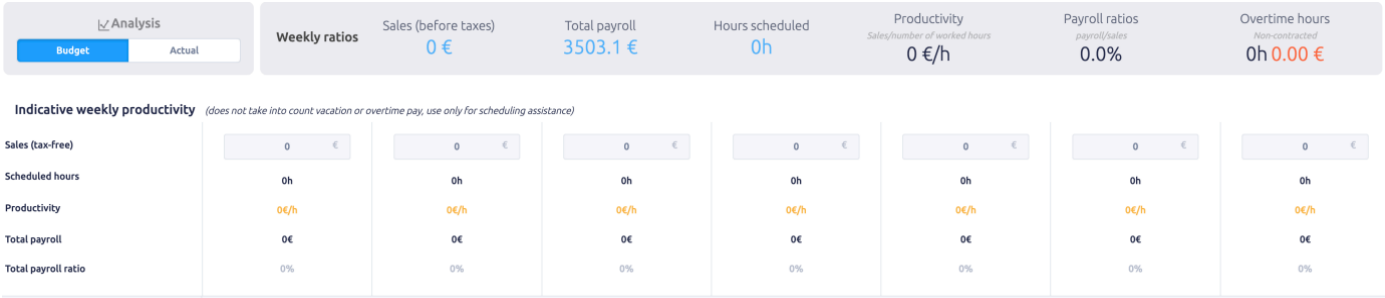
Response Code: 500

This can happen in case the start_date or end_date parameter does not respect the ISO 8601 format.

List Revenues

GET `/api/v1/revenues?week={week}`

The request information about revenues for the specified week for all locations. This can be used to get actual and estimated revenue values.



In the budget section you will see values for the estimated value and in the actual section you can get that for your actual budget. Both can be edited via the API.

Parameters

parameter	type	required	default	description
week	string	yes		The week in the ISO 8601 format.

HTTP Responses

Response Code: 200

Response Body:

```
[
  {
    "location_id": String, the location's partner id,
    "location_name": String, the location's name,
    "date": Date, the date of the revenue,
    "estimated_amount": Float, the estimated revenue,
    "actual_amount": Float, the actual revenue,
    "automated_update": Boolean, value that describes if the revenue will
    update automatically # TBD
  },
  ...
]
```

Response Code: 422

This can happen when you don't send all the mandatory parameters. In case you don't send week you will see this response code.

Response Body:

```
{
  "raw_error_messages": [
    "param is missing or the value is empty: week"
  ],
  "error": {
    "type": "missing_request_parameter",
    "parameter": "week",
    "codes": []
  },
  "message": "Missing parameter \"week\"."
}
```

Known bugs:

Response Code: 500

This can happen in case the week parameter does not respect the ISO 8601 format.

Create revenue

POST `/api/v1/revenues`

Request creates or updates the revenue. In case a revenue for the given location and date exists, it updates it. It returns the revenue information. Calls to the endpoint are subject to throttling.

Payload

```
{
  "location_id": String (required), the location's partner id,
  "date": String (required), the date for which the revenue is
  created, "amount": Float, the actual value of the revenue, decimal
  number
  "estimated_amount": Float, the estimated value fo the revenue, decimal
  number }
```

- At least one amount(actual or estimated) must be present.
- amount/estimated_amount expect a decimal number, in case anything else is expected, the value 0.0 is inserted

HTTP Responses

Response Code: `200`

Response Body:

```
{
  "location_id": String, the location's partner id,
  "location_name": String, the location's name,
  "date": Date, the date,
  "estimated_amount": Float, the estimated revenue,
  "actual_amount": Float, the actual revenue,
  "automated_update": Boolean, value that describes if the revenue will
  update automatically # TBD
}
```

Response Code: `404`

This can happen when the location id you have provided is not valid, meaning we couldn't find one in our database.

Response Body:

```
{
```

```
    "error": "Location ID is invalid"
  }
```

Response Code: 422

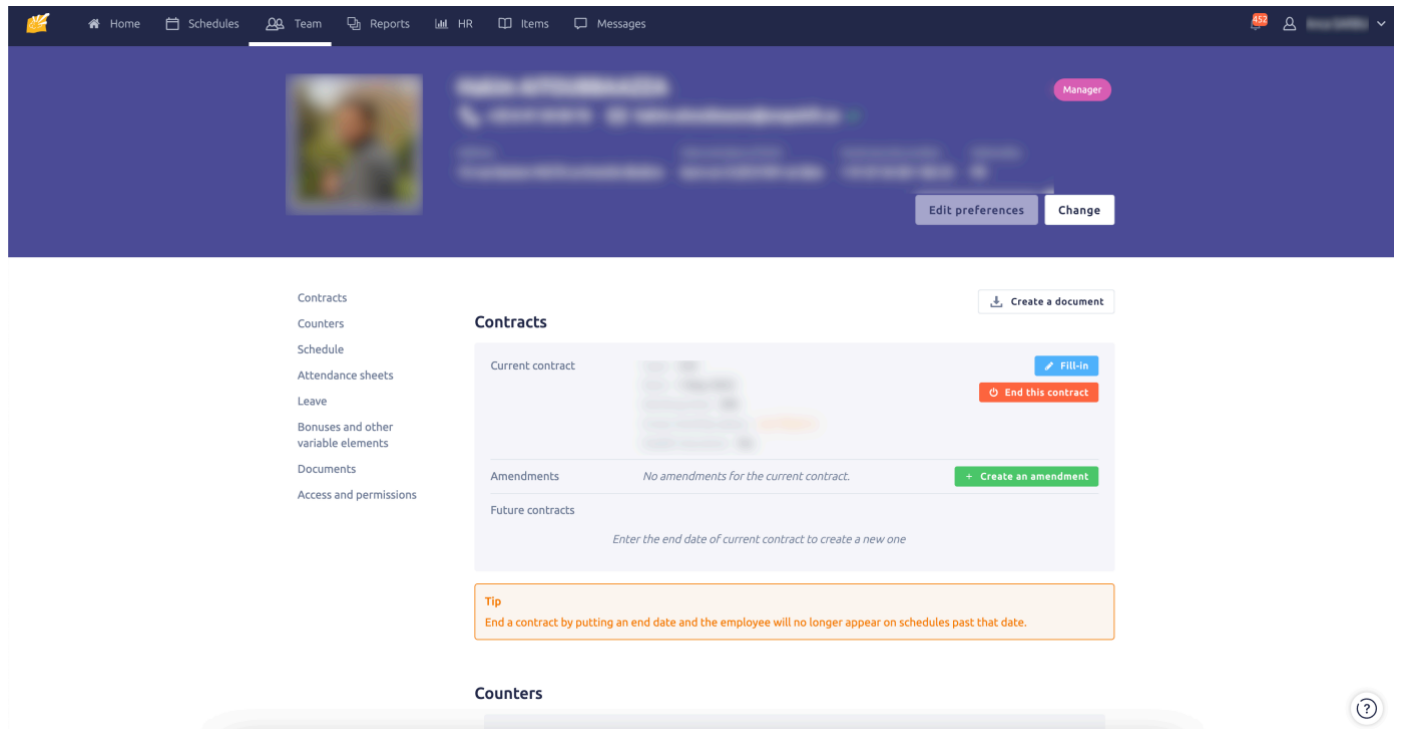
This can happen when your payload is invalid. An invalid payload is one that doesn't include the minimum required information (at least one value must be sent: `amount` or `estimated_amount`) or which has an invalid date value.

Response Body:

```
{
  "error": "At least one of 'amount' or 'estimated amount' must be
present" }
```

Single User

Request returns information about the user. This can be used to get detailed information about a single user in the account.

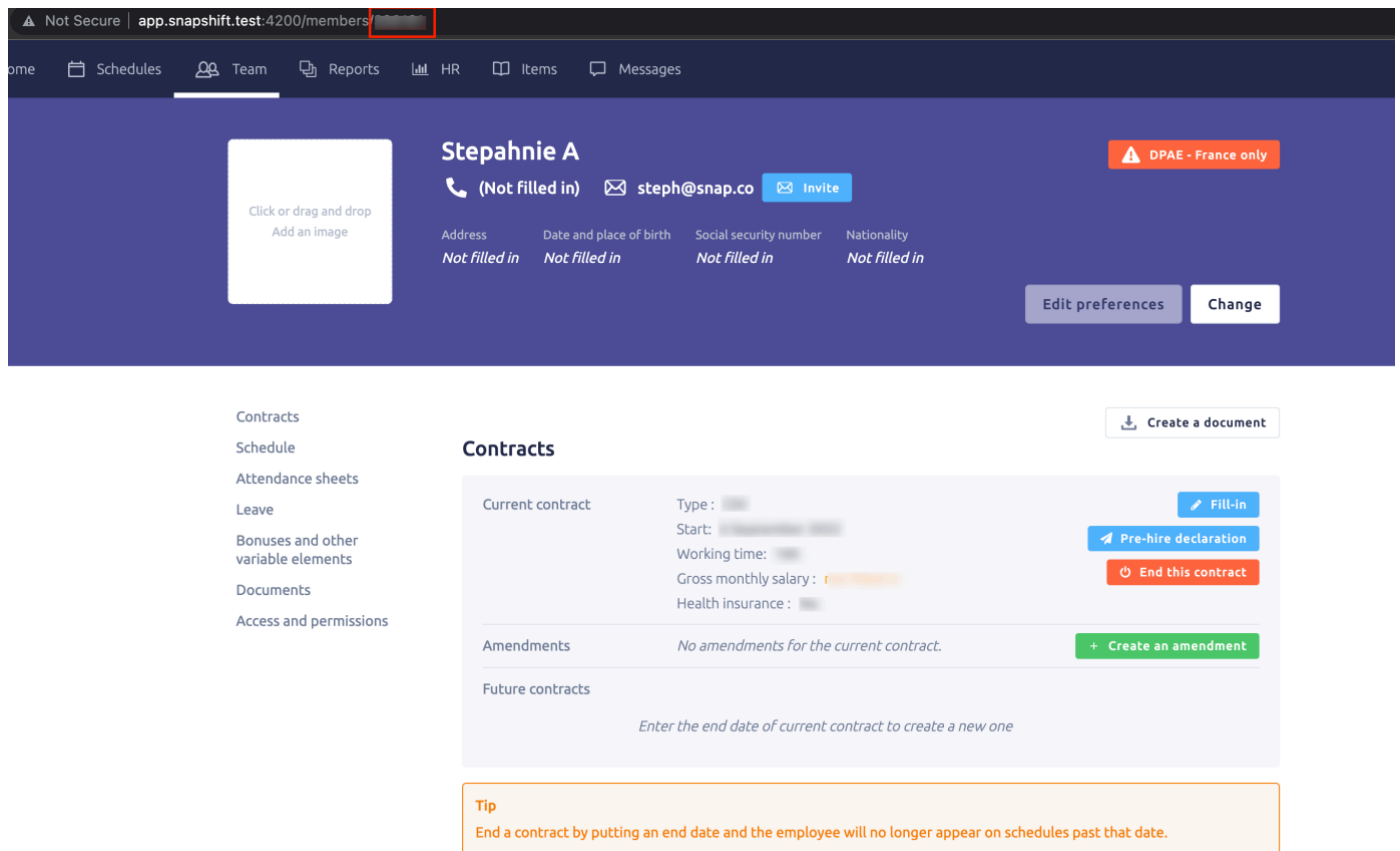


GET `/api/v1/users/{id}`

Parameters

parameter	type	required	default	description
id	integer	yes		The user id.

You can get the id of an user, by selecting the Team tab and selecting a user. The id will be visible in the url.



HTTP Responses

Response Code: 200

Response Body:

```
{
  "id": Integer, the user's id,
  "firstname": String, the user's first name,
  "lastname": String, the user's last name,
  "email": String, the user's email,
  "avatar_url": String, the url to the user's avatar,
  "memberships": Array[membership], the list of memberships
}
```

Definitions

These defines objects that were referenced in this entire documentation.

- custom value week:

```
{
  "2019-W18": Integer, value of days worked, # TBD
  "incomplete_week": Boolean
}
```

- event:

```
{
  "event_type": String, the event type,
  "event_occurred_at": Datetime, the time when the event
  happend, "note": String, note of the tevent
},
```

- location:

```
{
  "id": String, the partner id,
  "name": String, the location's name,
  "account_id": String, the account's partner id,
  "partner_id": String, the partner id,
  "snapshift_account_id": Integer, the account's id,
  "snapshift_location_id": Integer, the location's
  id, "teams": Array[team], list of teams
}
```

- membership:

```
{
  "id": Integer, the membership's id,
  "user_id": Integer, the user's id,
  "account_id": Integer, the user's account id,
  "employee_id": Integer, the user's employee id #
TBD }
```

- pad_user:

```
{
```

```
"id": String, the id,
"account_id": Integer, the account id,
"device_id": String, the device id,
"name": String, the name of the , # TBD
"team_ids": Array[Integer], list of the associated team
ids }
```

- team:

```
{
  "name": String, the team's name,
  "id": String, the team's partner id
}
```